Machine Learning I

Miguel García Moreno

2026-03-28

Contents

1 Section 1. K-means and agglomerative hierarchical clustering **3**

1.1 Data loading and preprocessing 3

1.2 Technique 1: K-means 4

1.2.1 Silhouette method 4

1.2.2 Model fit 5

1.2.3 Graphical representation 6

1.3 Technique 2: Agglomerative hierarchical clustering 6

1.3.1 Building the dendrogram 7

1.3.2 Model fit 8

1.4 Interpretation of results 8

2 Section 2. Weighted KNN and bootstrap 632+ **9**

2.1 Model fit 9

2.2 Generalisation ability via bootstrap 632+ 11

3 Section 3. Kernel Naive Bayes. Generalisation ability and AUC **13**

3.1 Model fit and evaluation of generalisation ability 13

3.2 Graphical representation 14

1 Section 1. K-means and agglomerative hierarchical clustering

Working with `datosred`, without using the `label` variable, apply and interpret two cluster-analysis techniques.

1.1 Data loading and preprocessing

```
# Package installation
paquetes<-c('ggplot2','cluster','factoextra','ks','kknn','cat','naivebayes','pROC')
for (i in 1:length(paquetes))
{
  if(require(paquetes[i],character.only = TRUE)) cat(paquetes[i], " loaded successfully\n")
  else {
    cat("Trying to install",paquetes[i])
    install.packages(paquetes[i])
    if(require(paquetes[i],character.only = TRUE)) cat(paquetes[i], " loaded successfully")
    else {
      stop("Could not install ", paquetes[i])
    }
  }
}
```

```
## ggplot2  loaded successfully

## cluster  loaded successfully

## factoextra  loaded successfully

## ks  loaded successfully

## kknn  loaded successfully

## cat  loaded successfully

## naivebayes  loaded successfully

## pROC  loaded successfully
```

```
# First five digits of the national ID (seed)
m <- 29509

# Load data-generation script
source("GenerarDatos.R")

# Inspect structure
str(datosred)
```

```
## 'data.frame':    75 obs. of  11 variables:
##  $ label   : Factor w/ 2 levels "Pullover","Ankle boot": 2 2 1 1 1 1 1 2 1 2 ...
##  $ pixel52 : num  201.7931 0.0691 0.2863 0.223 2.4833 ...
##  $ pixel81 : num  162.701 0.647 3.94 3.683 0.335 ...
##  $ pixel125: num  4.23 4.77e-03 1.83e+02 4.15e+01 2.43e+02 ...
##  $ pixel161: num  169.2 208.6 163.2 34.8 31.3 ...
##  $ pixel211: num  4.14 218.21 15.8 42.62 129.68 ...
##  $ pixel528: num  175 221.2 207.9 240 28.7 ...
##  $ pixel640: num  174 208 189 32 17 ...
##  $ pixel668: num  173.1 180.2 188.3 29.8 13.7 ...
##  $ pixel733: num  148.34 1.46 63.18 3.86 3.39 ...
##  $ pixel745: num  1.203 0.717 1.427 1.443 2.483 ...
```

We drop the *label* variable (it will not be used to build the clusters) and scale the features.

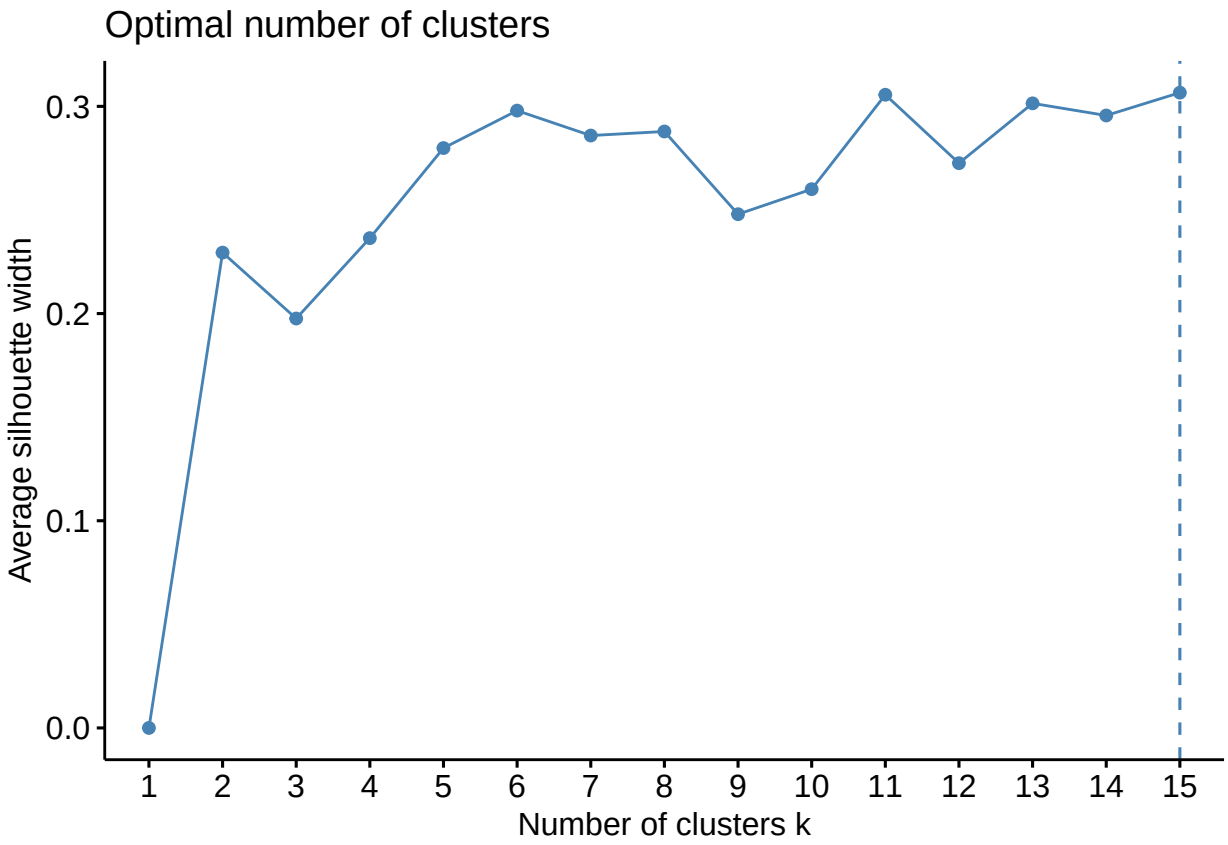
```
X <- datosred[, !(names(datosred) %in% "label")]
X_scaled <- scale(X)
```

1.2 Technique 1: K-means

We choose K-means as the first technique and select the model using the elbow / silhouette criterion.

1.2.1 Silhouette method

```
factoextra::fviz_nbclust(x = X_scaled, FUNcluster = kmeans, method = "silhouette", k.max = 15) +
  labs(title = "Optimal number of clusters")
```



If we did not know how many categories exist in the dataset, we could use the silhouette method to choose the optimal number of clusters. Here, knowing that there are two categories, we set K = 2. We also see that at k = 3 the average silhouette distance worsens slightly. We therefore fit the model with k = 2 and later compare the results with the true labels.

1.2.2 Model fit

```
km <- kmeans(X_scaled, centers = 2, nstart = 25)

# Size of each cluster
km$size
```

```
## [1] 36 39
```

```
# Cluster centres
km$centers
```

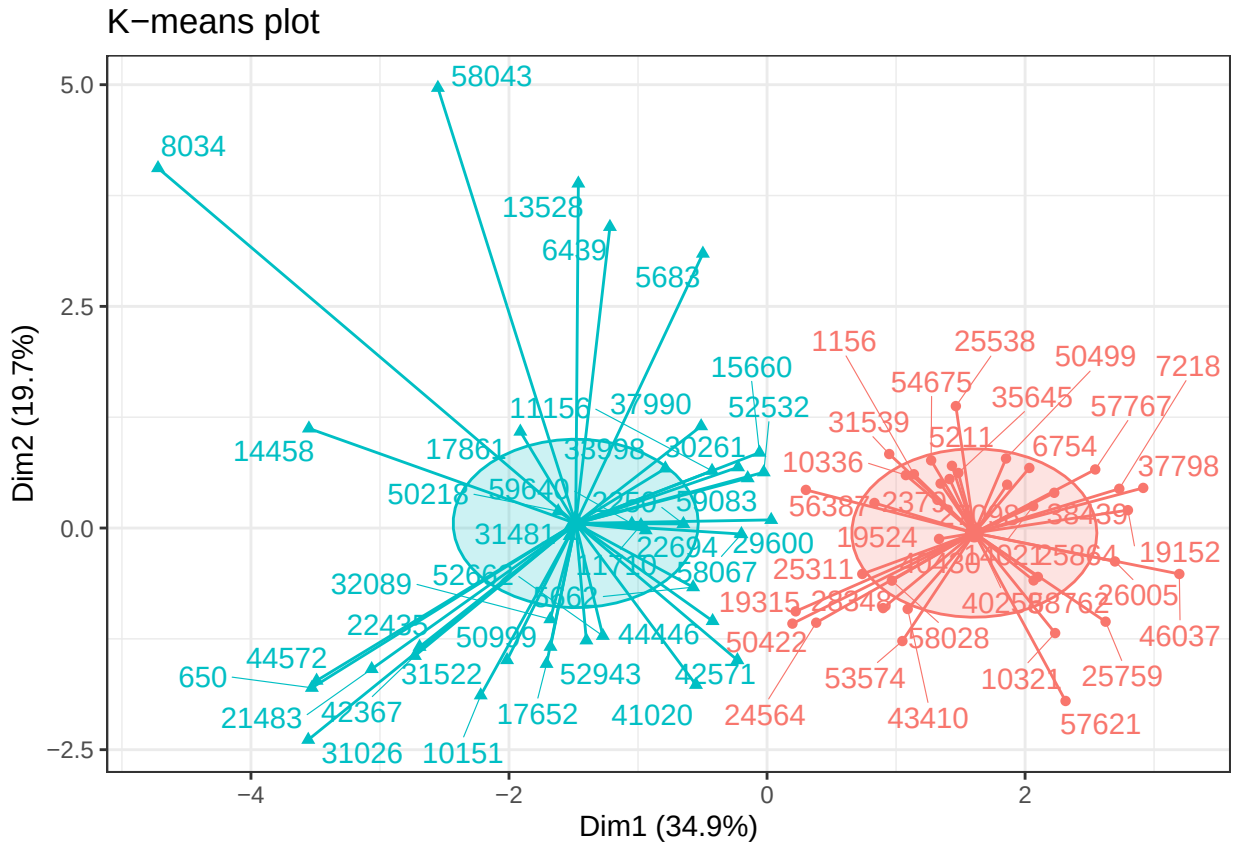
```
##      pixel52    pixel81   pixel125   pixel161   pixel211   pixel528   pixel640
## 1 -0.2392990 -0.2978319 -0.2379757 -0.8308488 -0.5653370 -0.5125409 -0.6386091
## 2  0.2208914  0.2749218  0.2196699  0.7669374  0.5218495  0.4731147  0.5894853
##     pixel668   pixel733   pixel745
## 1 -0.7309582 -0.4091260 -0.2809577
## 2  0.6747306  0.3776548  0.2593456
```

We add the classification to the data set so that we can later compare it with the true labels and assess the quality of the fit.

```
datosred_copy <- datosred
datosred_copy$cluster_kmeans <- as.factor(km$cluster)
```

1.2.3 Graphical representation

```
fviz_cluster(object = km, data = X_scaled, show.clust.cent = TRUE, ellipse.type = "euclid",
             star.plot = TRUE, repel = TRUE) + ggplot2::labs(title = "K-means plot") +
  theme_bw() + theme(legend.position = "none")
```



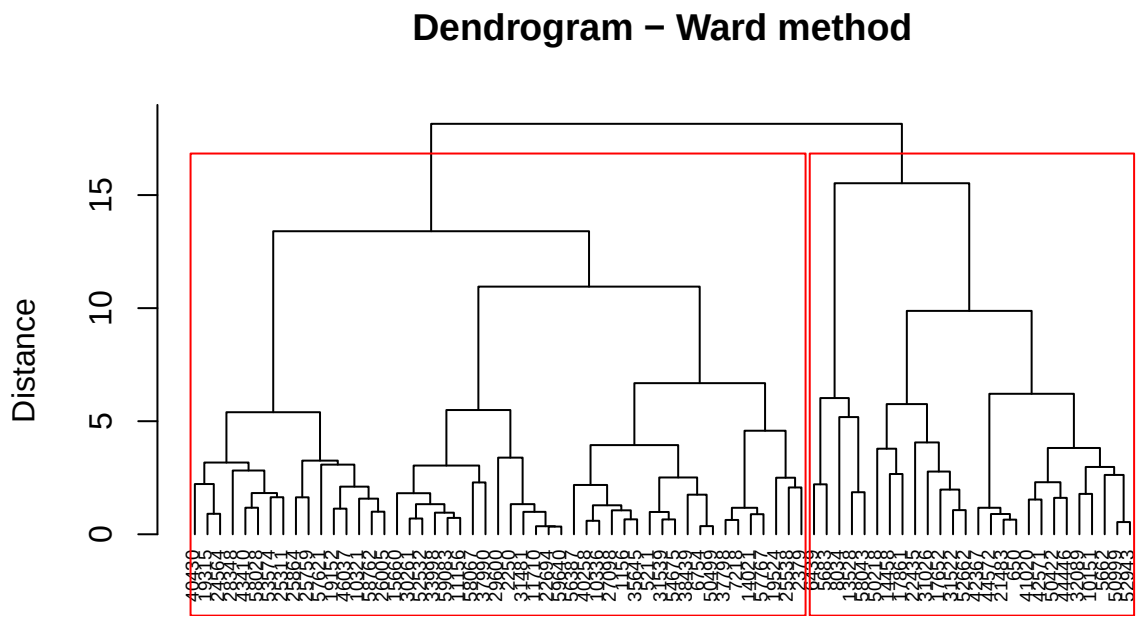
Because there are more than two dimensions, this plot shows each element's classification in terms of the first two principal components.

1.3 Technique 2: Agglomerative hierarchical clustering

As the second technique we use agglomerative hierarchical clustering.

1.3.1 Building the dendrogram

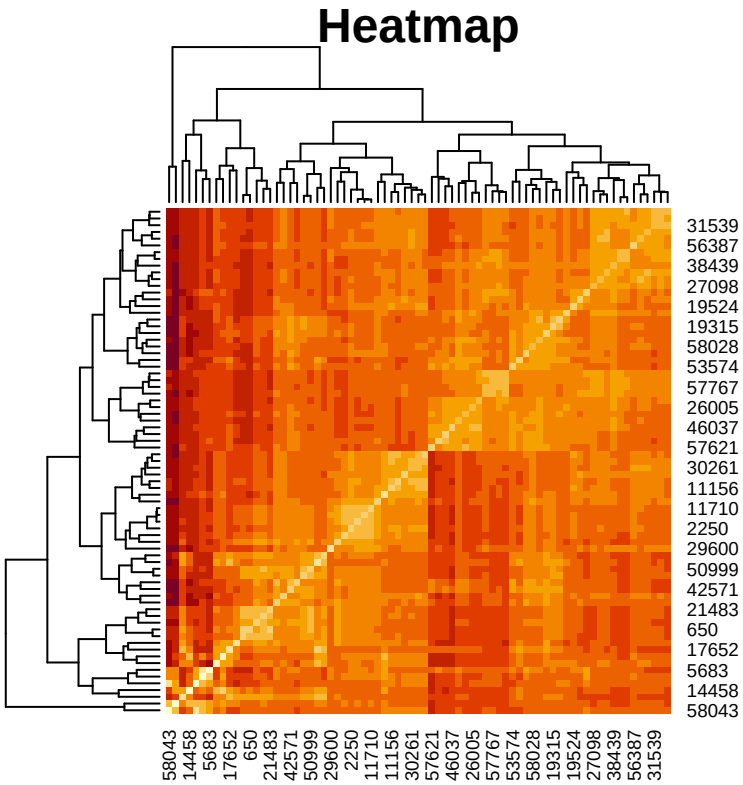
```
dist_matrix <- dist(X_scaled)
hc <- hclust(dist_matrix, method = "ward.D2")
plot(
  hc,
  main = "Dendrogram - Ward method",
  sub = "",
  xlab = "",
  ylab = "Distance",
  cex = 0.6,          # smaller labels
  hang = -1           # align leaves
)
rect.hclust(hc, k = 2, border = "red")
```



We show the dendrogram with Ward's method, which minimises within-cluster variance. Again we choose k = 2 for later comparison with the true labels. The red boxes highlight the two clusters obtained.

We also display the distances with a heatmap.

```
heatmap(as.matrix(dist_matrix),main = "Heatmap")
```



1.3.2 Model fit

We store the resulting classification in the data set so that we can later compare it with the true labels and assess the quality of the fit.

```
clusters_hc <- cutree(hc, k = 2)
datosred_copy$cluster_hier <- as.factor(clusters_hc)
```

1.4 Interpretation of results

Confusion matrices for each method:

```
levels(datosred_copy$cluster_kmeans) <- c("Pullover", "Ankle boot")
levels(datosred_copy$cluster_hier) <- c("Pullover", "Ankle boot")

#k-means
table(datosred_copy$cluster_kmeans, datosred_copy$label)
```

```
##
##              Pullover Ankle boot
##   Pullover         16         20
##   Ankle boot       15         24
```

```r
#Hierarchical clustering
table(datosred_copy$cluster_hier, datosred_copy$label)
```

```
##
##              Pullover Ankle boot
##   Pullover         16         10
##   Ankle boot       15         34
```

In both cases the classification is little better than chance. Hierarchical clustering shows a slightly better predictive ability.

---

2 Section 2. Weighted KNN and bootstrap 632+

Still using `datosred`, configure a weighted KNN model by cross-validation and estimate its generalisation ability with the bootstrap 632+ method.

Data preparation:

```r
set.seed(12345)

m <- dim(datosred)[1]  # number of cases

test <- sample(1:m, size = round(m/3), replace = FALSE,
               prob = rep(1/m, m))

# training set
x_train <- datosred[-test,]
# test set
x_test <- datosred[test,]
```

2.1 Model fit

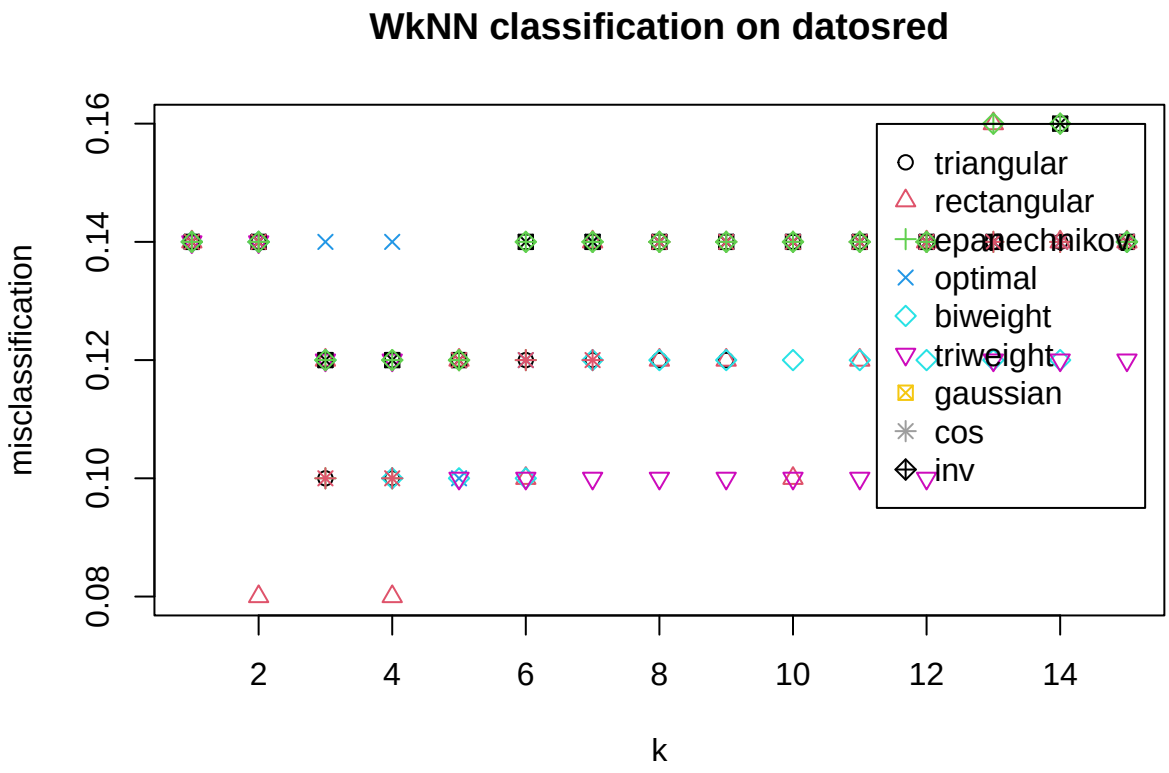
We fit the model by cross-validation.

```r
# Weighted KNN on the data
knn_model_cv <- train.kknn(label ~ ., x_train, kmax = 15,
                           kernel = c("triangular", "rectangular",
                           "epanechnikov", "optimal","biweight",
                           "triweight","gaussian","cos", "inv"),
                           distance = 1)
knn_model_cv
```

```
##
## Call:
## train.kknn(formula = label ~ ., data = x_train, kmax = 15, distance = 1,     kernel = c("triangular"
##
## Type of response variable: nominal
## Minimal misclassification: 0.08
## Best kernel: rectangular
## Best k: 2
```

The best model uses k = 2 neighbours and a rectangular kernel. Graphical summary of the results:

```
plot(knn_model_cv,main="WkNN classification on datosred")
```



The plot shows how the error rate evolves with the number of neighbours and the kernel type. Up to 6 neighbours the rectangular kernel performs best, and k = 2 is indeed optimal.

Confusion matrix:

```
fit_test <- predict(knn_model_cv, newdata = x_test)
confusion_matrix <- table("True_class"= x_test$label, "WkNN_class"= fit_test)
confusion_matrix
```

```
##             WkNN_class
## True_class   Pullover Ankle boot
##   Pullover          8          1
##   Ankle boot        1         15
```

```
prop.table(confusion_matrix)
```

```
##             WkNN_class
## True_class   Pullover Ankle boot
##   Pullover       0.32       0.04
##   Ankle boot     0.04       0.60
```

Results on the test set are good.

2.2 Generalisation ability via bootstrap 632+

First we compute the empirical error of the model fitted on the full training set.

```
best_knn_model = kknn(label ~ ., train=datosred, test = datosred, k=2,
                      kernel="rectangular", distance=1)

table(datosred$label, predict(best_knn_model))
```

```
##
##              Pullover Ankle boot
##   Pullover         31          0
##   Ankle boot        0         44
```

```
erroremp<- mean(datosred$label!=predict(best_knn_model))
cat("Empirical error =",100*erroremp ,"% \n")
```

```
## Empirical error = 0 %
```

We obtain a zero error. Next we build the bootstrap samples and their associated models, and compute the bootstrap and OOB errors.

```
B<- 2000 # number of bootstrap samples
set.seed(13579)

###### Variables ########
errorboot=numeric(B)
errorOOBb=numeric(B)
n= nrow(datosred)
Preditodos=matrix("",n,B)
IndicesOOB=matrix(0,n,B)
#########################


# Bootstrap loop
for (b in 1:B)
{
  if (b%%200==0) cat("Bootstrap sample number ",b,"\n")
# Generate bootstrap sample of indices
  indib<- sample(n,rep=T)
# Indices not included in indib
  indiOOB<-setdiff(1:n,indib)
# Fit model on the bootstrap sample
  boot_model<-kknn(label ~ ., train=datosred[indib,], test = datosred,
                   k=2, kernel="rectangular", distance=1)
# Predictions of this model on the training sample
  full_prediction = predict(boot_model)
# Error rate on the original sample
  errorboot[b]<- mean(full_prediction != datosred$label)
# Store the n predictions for model b
# as.character so category labels are stored instead of 1 and 2
  Preditodos[,b]=as.character(full_prediction)
```

```
  #print(preditodos[,b])
# Cases not included in bootstrap sample b
  IndicesOOB[indiOOB,b] =1
  errorOOBb<- mean(Preditodos[indiOOB,b] != datosred$label[indiOOB])
}
```

```
## Bootstrap sample number  200
## Bootstrap sample number  400
## Bootstrap sample number  600
## Bootstrap sample number  800
## Bootstrap sample number  1000
## Bootstrap sample number  1200
## Bootstrap sample number  1400
## Bootstrap sample number  1600
## Bootstrap sample number  1800
## Bootstrap sample number  2000
```

```
# Bootstrap and OOB error
errorB<- mean(errorboot)
errorOOB<- mean(errorOOBb)
```

We now compute the weighting coefficient w and finally the bootstrap 632+ error.

```
labels <- datosred$label
pred   <- best_knn_model$fitted.values

######### Variables #########
classes <- levels(labels)
noinf <- 0
#############################

# Loop to compute noinf
for (i in classes) {
  # Class proportion
  pi <- sum(labels == i) / n

  # Class-specific error
  qi <- sum(pred[labels == i] != i) / n

  noinf <- noinf + pi * (1 - qi)
}

# Weighting coefficient w
tsr = (errorOOB - erroremp) / (noinf - erroremp)
w = (0.632) / (1 - 0.368 * tsr)

# 632+ error
error632plus = (1 - w) * erroremp + w * errorOOB
error632plus
```

```
## [1] 0.07615681
```

The bootstrap 632+ error is 7.6%, which indicates that the model has good generalisation ability.

```
tsr
```

```
## [1] 0.1153846
```

The relative overfitting rate (tsr) is also very low.

3 Section 3. Kernel Naive Bayes. Generalisation ability and AUC

Next, split the 1000 cases in `datos` into training and test subsets (75% and 25%). Build a Naive Bayes model in which densities are estimated with the kernel method. Estimate generalisation ability via overall and per-class accuracy, as well as the ROC curve and AUC.

Data preparation:

```
# partition
set.seed(12345)
m <- dim(datos)[1]
train <- sample(1:m, size = round(0.75*m), replace = FALSE, prob = rep(1/m, m))
# training data set
test <- datos[-train,]
```

3.1 Model fit and evaluation of generalisation ability

We fit Naive Bayes with kernel density estimates, predict on the test set, and compute the confusion matrix.

```
model_nb <- naive_bayes(label ~ ., data = datos[train,],usekernel = TRUE)
pred_nb = predict(model_nb, test, type = "class")
table(test$label, pred_nb)
```

```
##             pred_nb
##              Pullover Ankle boot
##   Pullover        118          9
##   Ankle boot       17        106
```

The confusion matrix shows very good predictive performance.

Overall and per-class accuracy:

```
# overall accuracy
accuracy <- sum(pred_nb == test$label) / nrow(test)
cat("Overall accuracy:", round(accuracy * 100, 2), "%\n")
```

```
## Overall accuracy: 89.6 %
```

Around 90%. Because the data set is balanced, overall accuracy is a good indicator of generalisation ability—and this is a very strong result.

```r
# per-class accuracy
acierto_por_categoria <- prop.table(table(test$label, pred_nb), 1)
acierto_por_categoria_df <- as.data.frame(acierto_por_categoria)
colnames(acierto_por_categoria_df) <- c("Category", "Prediction", "Accuracy")
print(acierto_por_categoria_df)
```

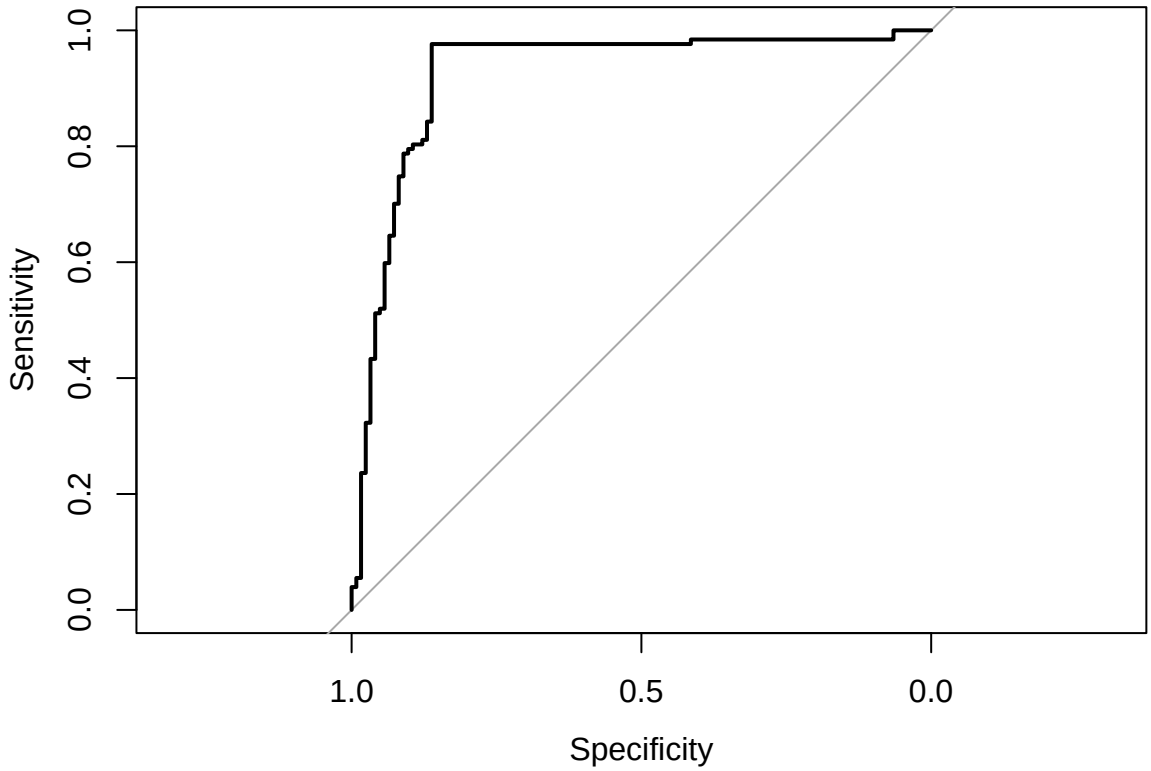
```
##     Category Prediction   Accuracy
## 1   Pullover   Pullover 0.92913386
## 2 Ankle boot   Pullover 0.13821138
## 3   Pullover Ankle boot 0.07086614
## 4 Ankle boot Ankle boot 0.86178862
```

For “Ankle boot” the model has a slightly harder time than for “Pullover”; even so, 86% is still a high rate.

3.2 Graphical representation

ROC curve:

```r
# ROC curve and AUC
# Prediction probabilities are needed for the ROC curve
pred_nb_prob <- predict(model_nb, test, type = "prob")
roc_obj <- roc(test$label, pred_nb_prob[, 2], levels = rev(levels(test$label)))
plot(roc_obj)
```



It is very strong—close to a perfect fit, almost an “L” shape.

```r
auc_value <- auc(roc_obj)
cat("AUC:", round(auc_value, 4), "\n")
```

```
## AUC: 0.9242
```

The AUC is very high, confirming that the model is good, in line with the earlier metrics and plots.